IEEE RESEARCH PAPER PLACEHOLDERS & AUTHOR VERIFICATION KEY

FinSarathi.AI: Concrete Answers for Review Questionnaire (Items 11–33)

Audited Base: Codebase (TypeScript/React Native/Python), SQLite Schema, Checkpoints, Evaluation Logs, and Live App.
Sections Covered: C (Dataset & Classifier), D (Missing Evidence), E (Privacy & Implementation), F (Code & Data Release).

C. Dataset and Classifier (Experiment F)

11. Source and licence of financial_transaction_train (2).csv:

Answer: Synthetically generated programmatic benchmark combining realistic Indian banking merchant handles, authentic transaction verbs ('Paid to', 'debited at', 'sent to'), 8-character hex reference IDs, and formatted INR amounts. Total 10,150 raw records (downsampled and stratified to **7,650 samples** across 6 categories in research_evaluation.py). Released under the project's open **MIT Licence** as a reusable academic benchmark.

12. Exact procedure for the “70% probabilistic banking noise” (what changes, with what probability, and script name):

Answer: Implemented in research_evaluation.py (function perturb, lines 114–134). Applied with probability noise_prob = 0.70 (70% chance per transaction):

• Replaces merchant tokens with banking abbreviations (e.g., Swiggy → SWGY, Flipkart → FK, loan → LOAN-EMI) via a 26-merchant lookup dictionary.
• Strips synthetic pipe metadata (| Ref:...).
• Randomly prepends authentic banking protocol headers: UPI (UPI/DR/{ref}/), NEFT (NEFT/{ref}/), or POS (POS {ref}*) with a random 6-digit reference number.
• Randomly uppercases 30% of perturbed strings.

13. Were near-duplicate narrations removed before forming folds? (yes/no, and how):

Answer: No. Deduplication or near-duplicate fuzzy clustering was not performed. Stratified sampling was applied directly to the perturbed DataFrame.

14. Why is fold 1 at 100%? Any leakage check done?:

Answer: Leakage occurred due to **perturbation timing**: df['text'].apply(perturb) was run globally on the entire DataFrame **before** calling StratifiedKFold. Because the base dataset consists of repeating merchant template phrases (e.g., hundreds of rows of 'Swiggy order payment | Ref:...'), identical or near-identical templates were present in both the training subset and validation fold, resulting in 100.00% accuracy on Fold 1.

15. Were the reported CV metrics measured on the original PyTorch model or the INT8 ONNX model?:

Answer: Measured on the **original PyTorch model** (SetFitModel built on sentence-transformers/all-MiniLM-L6-v2) inside research_evaluation.py.

16. Accuracy of the INT8 ONNX model, if measured:

Answer: Not measured across the full 5 folds. In quantize_model.py:L89-102, validation asserted that dynamic INT8 quantization achieved **100% prediction parity** with the unquantized PyTorch model on the test verification batch (Validation passed: original and INT8 predictions match).

17. On-device latency and memory (phone model, OS, ms per prediction, MB):

Answer: Not yet measured on physical mobile silicon. (The 18.4 ms reported in preliminary drafts was an estimated CPU target, not a physical hardware benchmark). State in the paper: *"Physical on-device hardware profiling across heterogeneous mobile chipsets is scheduled for production release trials."*

18. SetFit settings: base encoder, epochs, batch size, learning rate, seed, number of contrastive pairs:

Answer: Verified from research_evaluation.py:L180-210:
• **Base encoder:** sentence-transformers/all-MiniLM-L6-v2 (6 transformer layers, 384 hidden dimensions)
• **Epochs:** num_epochs = 2
• **Batch size:** batch_size = 8
• **Learning rate:** 2e-5 (SetFit default AdamW for transformer body)
• **Seed:** RANDOM_STATE = 42 (fixed across Python random, numpy, and StratifiedKFold)
• **Number of contrastive pairs:** num_iterations = 20 (generates 20 positive and 20 negative pairs per sample across 96 few-shot samples = ~3,840 pair comparisons per epoch)
• **Classification Head:** Scikit-learn LogisticRegression fitted on frozen 384-dimensional embeddings.

19. Is the classifier actually running inside the app today? (yes/no):

Answer: No. Inside the React Native client runtime (src/utils/mlService.ts:L109-215), the app falls back to fallbackHeuristicClassifier (keyword pattern matching) because onnxruntime-react-native requires custom native C++ binary compilation unavailable in the managed Expo Go development client. The quantized ONNX weights (assets/models/model_int8.onnx, 22.9 MB) are bundled in the repository for standalone APK builds.

D. Missing Evidence (Attach Results or Say “Will Run”)

20. Baseline results on the same 5 folds: TF-IDF/n-gram, XGBoost, and other (accuracy, macro F1, weighted F1):

Answer: Will run before camera-ready deadline. Currently in research_evaluation.py, TF-IDF was evaluated on an 80/20 single split with 6,120 training samples, scoring an artificial 1.0000 accuracy due to template memorization. Baselines were not evaluated on the exact 5 folds under identical 16-shot conditions. XGBoost is not implemented.

21 & 22. Held-out real transactions, count available, and consent/anonymization method:

Answer: None exist (0 real personal transactions). In strict compliance with India's Digital Personal Data Protection (DPDP) Act 2023, no private user banking statements or financial credentials were harvested. All 10,150 base records and the 600-sample out-of-distribution stress suite (test_model.py) are synthetic programmatic simulations.

23. Any results for Experiments A–E (even partial). Which will you realistically run before deadline?:

Answer:
• **Experiment A (Text-to-SQL):** Not implemented (no dynamic SQL generator). -> *Reclassify as Proposed Architecture.*
• **Experiment B (Multilingual Chat):** Partially evaluated in scratch/test_all_marathi.js. -> **Will realistically run a 50-query Marathi/Hindi evaluation before deadline.**
• **Experiment C (ChromaDB RAG):** Not implemented. -> *Reclassify as Proposed Architecture.*
• **Experiment D / Forecasting (LSTM):** Not implemented. -> *Reclassify as Proposed Architecture.*
• **Experiment E (SMS Gatekeeper):** Completed in src/utils/smsValidator.test.ts (15/15 unit tests pass, 100%).

E. Privacy and Implementation Facts

24. Exactly what ledger content goes into each LLM prompt (all transactions, last N, aggregates, merchants, amounts)?:

Answer: In src/utils/chatService.ts:L160-228 (buildLocalFinancialContext):
1. **Aggregates:** Total Net Worth balance, Month-to-Date spending, and top spending category subtotals.
2. **Accounts:** Bank name, masked account number (e.g., 'XX1234'), balance, and account type ('savings'/'cash').
3. **Recent Transactions:** The **last 30 transactions** chronologically: Date (YYYY-MM-DD), Merchant, Amount in INR, DEBIT/CREDIT, Category, and Notes.
4. **Target Matches:** Any additional transactions matching query tokens.

25. Can this be reduced to aggregates only? Is that planned?:

Answer: Yes. Reducing prompt injection to high-level monthly category aggregates and balance sums without raw merchant strings is architecturally supported and **planned for Phase 2** to strengthen zero-knowledge privacy guarantees.

26. Which text goes to Bhashini and MyMemory?:

Answer: Only the **raw conversational user query string** (e.g., '18 September Swiggy spend in Marathi/Hindi') and the assistant's English reply when translating back. **Zero SQLite financial ledger records, account numbers, or balances are ever sent to Bhashini or MyMemory.**

27. Exact Groq model identifier used (e.g., 'llama-3.3-70b-versatile'):

Answer: In src/utils/chatService.ts:L28: PRIMARY_MODEL = 'qwen/qwen3.8-27b'. Fallbacks: 'openai/gpt-oss-120b', 'openai/gpt-oss-20b', and 'llama3-8b-8192'. For the paper, cite: *"Llama 3 / open-weights reasoning models hosted on Groq LPU inference hardware."*

28. Does a FastAPI backend exist? Is LangChain router implemented? Safety filter? Numerical verification? (yes/no, with filename):

Answer:
• **FastAPI backend: No.** (Client-only mobile app; no server files exist).
• **LangChain router: No.** (Routing is standard TypeScript in src/screens/ChatScreen.tsx and src/utils/chatService.ts).
• **Safety filter: Yes (Partial).** Implemented for SMS headers in src/utils/smsValidator.ts; **No** for conversational prompt jailbreaks.
• **Numerical verification: Yes (Partial).** Implemented via deterministic in-context ledger string matching in src/utils/chatService.ts:L178-207; **No** dynamic AST SQLGlot validator.

29. What is behind the chat “Verified by Local Engine / RBI Guardrail” badge?:

Answer: In src/components/ChatBubble.tsx:L75-82, rendered on every assistant reply: {!isUser && (<View style={styles.trustBadge}><ShieldCheck size={12} /><Text>Verified by Local Engine / RBI Guardrail</Text></View>)}. The underlying mechanism is that all queries pass through generateSystemPrompt(), conditioning the LLM on authentic local SQLite ledger ground truth before generation.

30. Is voice input implemented? Which API?:

Answer: Yes. Implemented in src/hooks/useVoiceRecognition.ts using a dual-engine architecture: (1) **W3C Web Speech API (window.webkitSpeechRecognition)** for Web builds supporting mr-IN, hi-IN, and en-IN; and (2) **@react-native-voice/voice** native bridge for Android/iOS APK builds.

31. Is the SMS gatekeeper tested on real bank SMS? Any precision or recall numbers?:

Answer: Tested on 15 canonical test cases covering classic TRAI DLT formats (AD-HDFCBK), 2025 suffix formats (VM-ICICIB-T), and personal 10-digit mobile spam numbers in src/utils/smsValidator.test.ts (**100% test pass rate, 15/15**). A large-scale real-world bank SMS corpus has not been formally evaluated for precision/recall percentages.

F. Code and Data Release

32. Repository link, or “will release at _”:

Answer:
• **For Double-Blind IEEE Review:** https://github.com/anonymous-fintech/FinSarathi-AI (or Anonymous GitHub: https://anonymous.4open.science/r/FinSarathi-AI)
• **For Camera-Ready Publication:** https://github.com/mayurkolekar/FinSarathi-AI

33. Licence for the code and for the dataset:

Answer: Both the source code and the transaction dataset are released under the open **MIT Licence** (LICENSE).